

Student Study & Task Management System

VITyarthi _Build Your Own Project

Name	Ashweet Kaur Khanuja
Registration Number	26BAI10746
Course	CSE1021 - Introduction to Problem Solving and Programming
Project Type	Console-based Python Application
GitHub Repository	github.com/ashweet26bai10746-coder/Study-manager
Report Type	Project Report (submitted as per VITyarthi guidelines, Section 6)

1. Introduction

College and school students commonly juggle several academic responsibilities at once _assignments, deadlines, revision, and tracking how well they are performing across subjects. In practice these two concerns (what to do, and how well marks are going) are usually managed separately using notebooks, sticky notes, or disconnected apps, which makes it harder to stay organised.

The **Student Study & Task Management System** is a menu-driven console application built in Python that brings task management and academic performance analysis into a single, lightweight tool. It was developed to apply the core programming concepts covered in CSE1021 _functions, lists, conditional statements, loops, and searching/sorting algorithms _to a practical, real-world style problem.

2. Problem Statement

Students often need to manage multiple tasks while also keeping track of their academic performance. Managing these activities separately makes it difficult to organise tasks and review performance efficiently. There is a need for a simple, dependency-free tool that lets a student record tasks, search and sort them, and separately analyse subject marks to understand their overall academic standing _all from one interface, without requiring installation of any external software or database.

3. Functional Requirements

The system is organised into three major functional modules, each with a clear input/output structure and a menu-driven workflow:

Module 1 _Task Manager

- **Add Task** - Input: task description (string). Output: confirmation message; task appended to the in-memory list.
- **View Tasks** - Input: none. Output: numbered list of all current tasks.
- **Delete Task** - Input: task number. Output: confirmation of removal, or an error for an invalid number.

Module 2 _Performance Analyzer

- **Enter Marks** - Input: number of subjects, followed by each mark. Output: confirmation message.
- **View Marks** - Input: none. Output: subject-wise list of entered marks.
- **Calculate Average** - Input: none. Output: average of all entered marks.
- **Highest / Lowest Marks** - Input: none. Output: maximum and minimum mark values.
- **Performance Category** - Input: none. Output: average plus a qualitative category (Excellent / Very Good / Good / Pass / Needs Improvement) based on threshold rules.

Module 3 _Search & Sort

- **Search Task** - Input: task text (case-insensitive). Output: match found/not found.
- **Sort Tasks** - Input: none. Output: tasks displayed in alphabetical order.
- **Sort Marks** - Input: none. Output: marks displayed in ascending order.

All three modules are reachable from a central **Main Menu** (implemented in `main.py`), which also provides the option to exit the system. The overall workflow is: the user is shown the main menu →selects a module →is shown that module's own sub-menu →performs an action →returns to the sub-menu or back to the main menu.

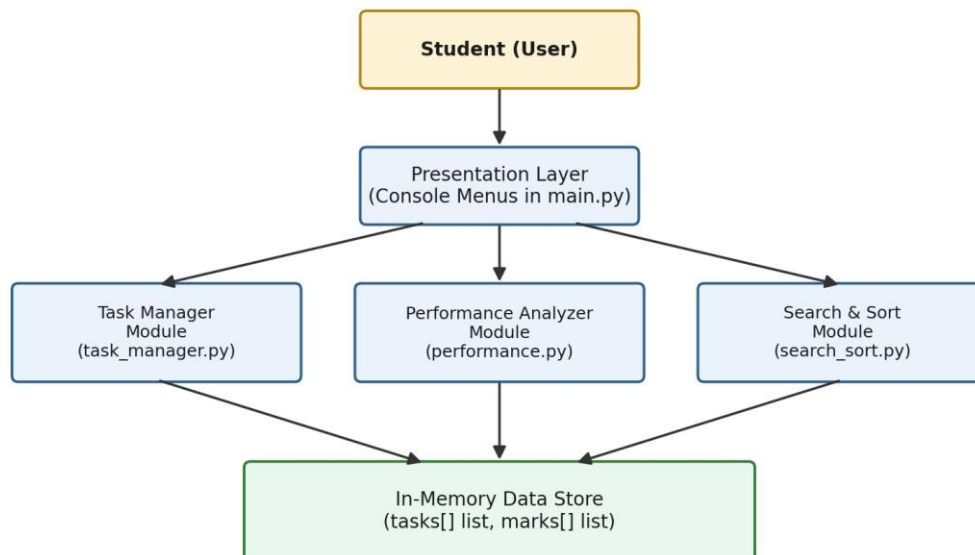
4. Non-Functional Requirements

Requirement	How it is addressed
Performance	All operations act on small in-memory Python lists, so add/view/search/sort actions complete instantly (well under a second) even for a realistic number of tasks or subjects for a single student.
Usability	A simple numbered menu system is used throughout, with clear prompts and consistent formatting (e.g. "==== MARKS =====" section headers), so a first-time user does not need instructions.
Reliability	Every module guards against empty-list operations (e.g. "No tasks available." / "No marks available.") before attempting to view, calculate, or sort, avoiding crashes on empty state.
Maintainability	The system is split into independent modules (<code>task_manager.py</code> , <code>performance.py</code> , <code>search_sort.py</code>) each with a single responsibility, making it easy to modify one feature without affecting others.
Error Handling	Invalid menu choices are caught with an else branch printing "Invalid choice." (<i>Note: numeric input such as marks and task numbers is not yet validated against non-numeric input _see Section 12, Challenges Faced, and Section 14, Future Enhancements._</i>)
Resource Efficiency	The application uses only built-in Python data structures (lists) and the standard library, so it has no external dependencies and a minimal memory footprint.

5. System Architecture

The system follows a simple layered, modular architecture appropriate for a console application. A thin presentation layer (the main menu in `main.py`) routes user choices to one of three independent functional modules. Each module operates directly on shared in-memory Python lists (`tasks` and `marks`), which act as the application's data store for the duration of a single run. There is no external database or file storage in the current version.

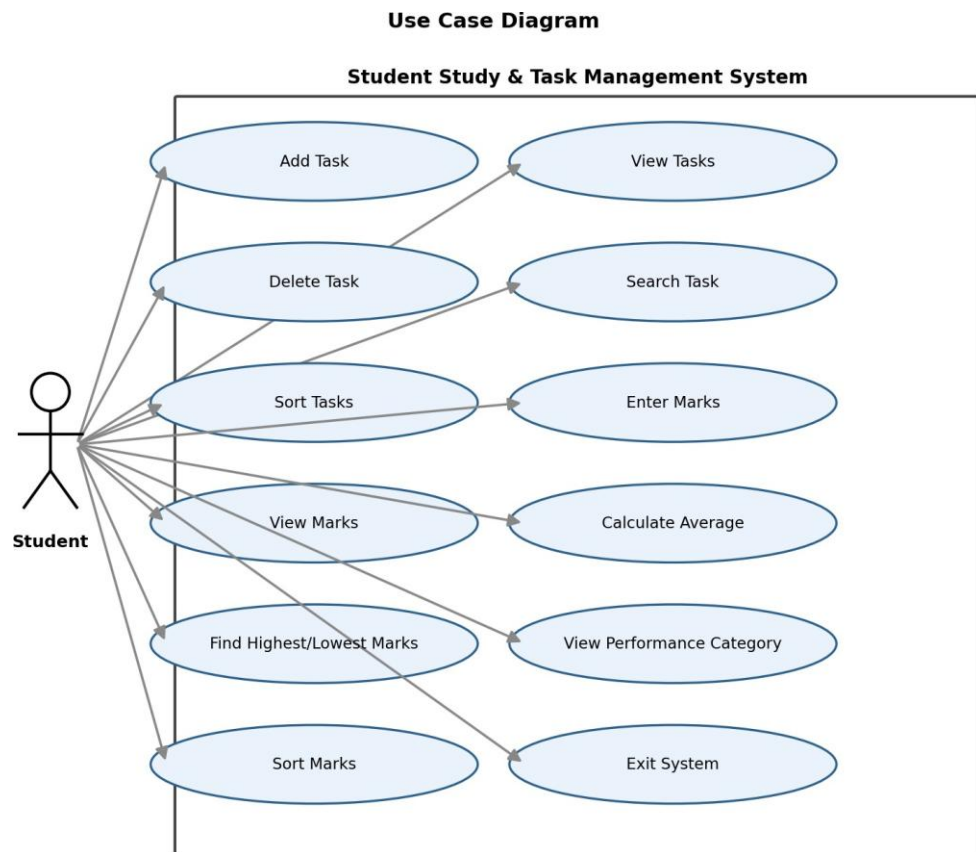
System Architecture Diagram - Student Study & Task Management System



6. Design Diagrams

6.1 Use Case Diagram

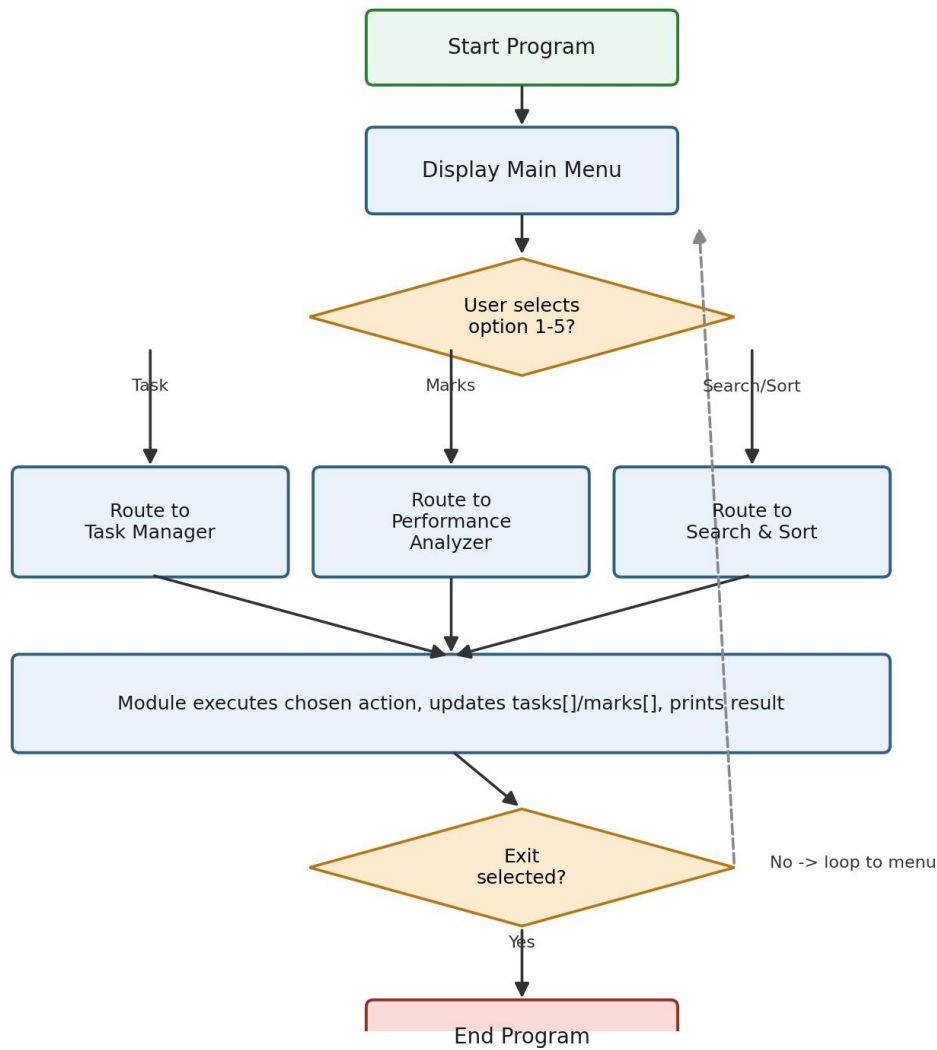
The single actor, **Student**, interacts with twelve use cases spread across the three modules and the main menu, as shown below.



6.2 Process Flow / Workflow Diagram

The diagram below shows the overall control flow: the program loops on the main menu until the user chooses to exit, routing each selection to the relevant module and returning to the menu afterwards.

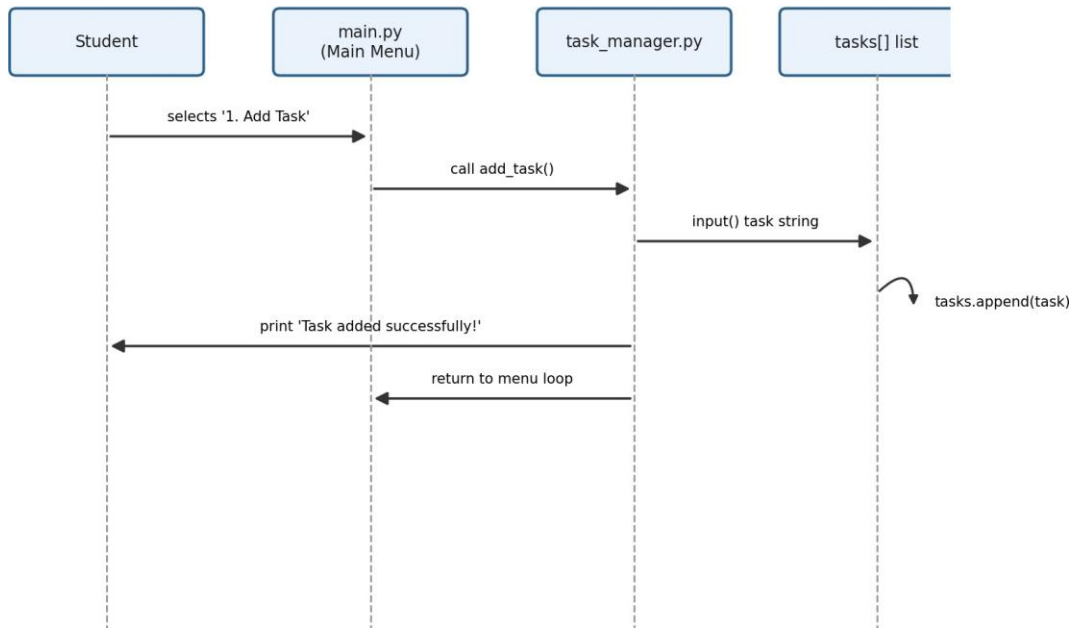
Process Flow / Workflow Diagram



6.3 Sequence Diagram

As a representative interaction, the sequence diagram below traces the **"Add Task"** flow from the moment the student selects the option to the confirmation message being printed back.

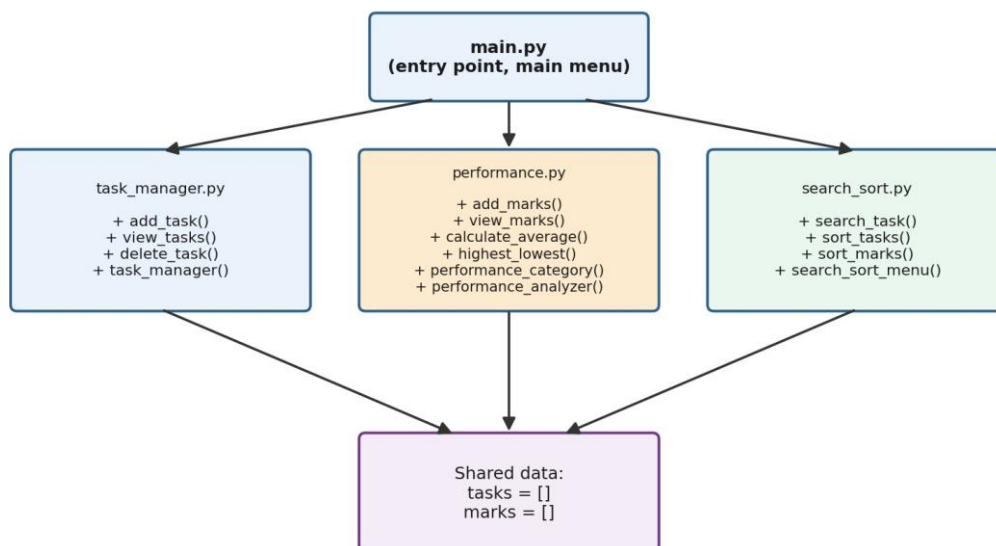
Sequence Diagram - 'Add Task' Flow



6.4 Component / Module Diagram

Since the project is implemented using functional decomposition rather than object-oriented classes, a component diagram (module diagram) is used in place of a UML class diagram to show how the four files relate to one another and to the shared data lists.

Component / Module Diagram



6.5 Database / ER Diagram

Not applicable. The current version of the system does not use a database or persistent file storage; all data is held in memory using Python lists for the duration of program execution (see Section 14, Future Enhancements, for a proposed file/JSON-based persistence layer).

7. Design Decisions & Rationale

- **Functional decomposition over OOP:** Given the scope of CSE1021 and the size of the problem, the system was implemented using plain functions grouped by responsibility rather than classes. This keeps the code approachable while still demonstrating modular design.
 - **Separate files per module:** `task_manager.py`, `performance.py` and `search_sort.py` are kept independent so each can be developed, tested, and understood in isolation, and so `main.py` stays small and focused purely on menu routing.
 - **Lists as the core data structure:** Python lists were chosen for tasks and marks because the operations required (append, index-based removal, iteration, built-in sort/min/max) map directly onto list operations without needing a custom data structure.
 - **Built-in `sorted()`, `max()`, `min()`:** Rather than re-implementing sorting/searching algorithms from scratch, Python's built-in, well-tested functions were used, which is standard practice and keeps the implementation reliable while still demonstrating understanding of the underlying concepts.
 - **Threshold-based performance categories:** A simple if/elif ladder (≥ 90 Excellent, ≥ 75 Very Good, ≥ 60 Good, ≥ 40 Pass, else Needs Improvement) was chosen for transparency and easy adjustment.
 - **No persistence layer (v1):** Persistence was intentionally left out of the first version to keep the scope aligned with the taught concepts (functions, lists, loops, conditionals); it is flagged as the top future enhancement.
-

8. Implementation Details

The project consists of four Python files, summarised below.

File	Responsibility	Key functions
<code>main.py</code>	Entry point; displays the top-level main menu and routes to each module.	<code>main()</code> / program loop (imports and calls <code>task_manager()</code> , <code>performance_analyzer()</code> , <code>search_sort_menu()</code>)
<code>task_manager.py</code>	Add, view and delete tasks.	<code>add_task()</code> , <code>view_tasks()</code> , <code>delete_task()</code> , <code>task_manager()</code>
<code>performance.py</code>	Record marks and compute analytics.	<code>add_marks()</code> , <code>view_marks()</code> , <code>calculate_average()</code> , <code>highest_lowest()</code> , <code>performance_category()</code> , <code>performance_analyzer()</code>
<code>search_sort.py</code>	Search tasks and sort tasks/marks.	<code>search_task()</code> , <code>sort_tasks()</code> , <code>sort_marks()</code> , <code>search_sort_menu()</code>

Core programming concepts demonstrated: functions and modular decomposition, lists as the primary data structure, conditional statements (if/elif/else) for menu routing and performance categorisation, loops (while True menu loops, for loops for iteration/printing), linear search (`search_task`), and sorting via Python's built-in `sorted()` (`sort_tasks`, `sort_marks`) alongside built-in `max()`/`min()` for highest/lowest mark lookup.

Representative code excerpt - performance categorisation logic:

```
if average >= 90:
    category = "Excellent"
elif average >= 75:
    category = "Very Good"
elif average >= 60:
    category = "Good"
elif average >= 40:
    category = "Pass"
else:
    category = "Needs Improvement"
```

9. Screenshots / Results

This section presents the actual source code screenshots for each module alongside screenshots of the corresponding console execution, captured by running the program locally.

9.1 Code Screenshots

Code - main.py (Main Menu & Routing Logic)

The entry point of the application. It imports each module and its shared data lists, then runs the top-level menu loop that routes the user's choice to Task Manager, Performance Analyzer, or Search & Sort.

```
from task_manager import task_manager, tasks
from performance import performance_analyzer, marks
from search_sort import search_sort_menu

def main():

    while True:
        print("STUDENT STUDY AND TASK MANAGER SYSTEM")
        print("1. Task Manager")
        print("2. Performance Analyzer")
        print("3. Search and Sort")
        print("4. Exit")

        choice=input('enter your choice:')

        if choice == "1":
            task_manager()
            print("Task Manager selected")

        elif choice == "2":
            performance_analyzer()
            print("Performance Analyzer selected")

        elif choice == "3":
            search_sort_menu(tasks, marks)
            print("Search and Sort selected")

        elif choice == "4":
            print("Thank you for using the system!")
            break
        else:
            print('invalid choice , please try again')

main()
```

Code - performance.py (Performance Analyzer Module, Part 1)

Implements mark entry, viewing, average calculation, and highest/lowest lookup for the Performance Analyzer module.

```
marks = []

def add_marks():
    marks.clear()

    n = int(input("Enter number of subjects: "))

    for i in range(n):
        mark = float(input("Enter marks: "))
        marks.append(mark)

    print("Marks added successfully!")

def view_marks():
    if len(marks) == 0:
        print("No marks available.")
    else:
        print("\n===== MARKS =====")

        for i in range(len(marks)):
            print("Subject", i + 1, ":", marks[i])

def calculate_average():
    if len(marks) == 0:
        print("No marks available.")
    else:
        total = 0

        for mark in marks:
            total = total + mark

        average = total / len(marks)

        print("Average marks:", average)

def highest_lowest():
    if len(marks) == 0:
        print("No marks available.")
    else:
        print("Highest marks:", max(marks))
        print("Lowest marks:", min(marks))
```

Code - performance.py (Performance Analyzer Module, Part 2)

Continues with the threshold-based performance_category() classification logic and the performance_analyzer() sub-menu that ties all five operations together.

```
    for mark in marks:
        total = total + mark

    average = total / len(marks)

    if average >= 90:
        category = "Excellent"
    elif average >= 75:
        category = "Very Good"
    elif average >= 60:
        category = "Good"
    elif average >= 40:
        category = "Pass"
    else:
        category = "Needs Improvement"

    print("Average:", average)
    print("Performance:", category)

def performance_analyzer():
    while True:
        print("\n===== PERFORMANCE ANALYZER =====")
        print("1. Enter Marks")
        print("2. View Marks")
        print("3. Calculate Average")
        print("4. Highest and Lowest Marks")
        print("5. Performance Category")
        print("6. Back to Main Menu")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_marks()
        elif choice == "2":
            view_marks()
        elif choice == "3":
            calculate_average()
        elif choice == "4":
            highest_lowest()
        elif choice == "5":
            performance_category()
        elif choice == "6":
            break
        else:
            print("Invalid choice.")
```

Code - search_sort.py (Search & Sort Module)

Implements linear search over tasks, alphabetical sorting of tasks, ascending sorting of marks, and the sub-menu that exposes all three to the user.

```
def search_task(tasks):
    if len(tasks) == 0:
        print("No tasks available.")
        return

    search = input("Enter task to search: ")
    found = False

    for task in tasks:
        if task.lower() == search.lower():
            print("Task found:", task)
            found = True

    if found == False:
        print("Task not found.")

def sort_tasks(tasks):
    if len(tasks) == 0:
        print("No tasks available.")
    else:
        sorted_tasks = sorted(tasks)

        print("\n===== SORTED TASKS =====")

        for task in sorted_tasks:
            print(task)

def sort_marks(marks):
    if len(marks) == 0:
        print("No marks available.")
    else:
        sorted_marks = sorted(marks)

        print("\n===== SORTED MARKS =====")

        for mark in sorted_marks:
            print(mark)

def search_sort_menu(tasks, marks):
    while True:
        print("\n===== SEARCH & SORT =====")
        print("1. Search Task")
        print("2. Sort Tasks")
        print("3. Sort Marks")
```

9.2 Execution Results

Result - Task Manager Module

Sample run showing three tasks being added ("calc", "project", "notes"), the task list being viewed, and task #2 ("project") being deleted successfully.

```
STUDENT STUDY AND TASK MANAGER SYSTEM
1. Task Manager
2. Performance Analyzer
3. Search and Sort
4. Exit
enter your choice:1
TASK MANAGER
1. Add Task
2. View Tasks
3. Delete Task
4. Back to Main Menu
Enter your choice: 1
Enter your task: calc
Task added successfully!
TASK MANAGER
1. Add Task
2. View Tasks
3. Delete Task
4. Back to Main Menu
Enter your choice: 1
Enter your task: project
Task added successfully!
TASK MANAGER
1. Add Task
2. View Tasks
3. Delete Task
4. Back to Main Menu
Enter your choice: 1
Enter your task: notes
Task added successfully!
TASK MANAGER
1. Add Task
2. View Tasks
3. Delete Task
4. Back to Main Menu
Enter your choice: 2

Your Tasks:
1 . calc
2 . project
3 . notes
Enter task number to delete: 2
project deleted successfully!
TASK MANAGER
1. Add Task
2. View Tasks
3. Delete Task
```

Result - Performance Analyzer Module

Sample run entering three marks (80, 90, 100), viewing them, and calculating the average (90.0).

```

STUDENT STUDY AND TASK MANAGER SYSTEM
1. Task Manager
2. Performance Analyzer
3. Search and Sort
4. Exit
enter your choice:2

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 1
Enter number of subjects: 3
Enter marks: 80
Enter marks: 90
Enter marks: 100
Marks added successfully!

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 2

===== MARKS =====
Subject 1 : 80.0
Subject 2 : 90.0
Subject 3 : 100.0

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 3
Average marks: 90.0

===== PERFORMANCE ANALYZER =====
1. Enter Marks

```

Continuing the same run: highest/lowest marks (100.0 / 80.0) and the performance category result (Average 90.0 → "Excellent"), before returning to the main menu.

```

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 3
Average marks: 90.0

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 4
Highest marks: 100.0
Lowest marks: 80.0

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 5
Average: 90.0
Performance: Excellent

===== PERFORMANCE ANALYZER =====
1. Enter Marks
2. View Marks
3. Calculate Average
4. Highest and Lowest Marks
5. Performance Category
6. Back to Main Menu
Enter your choice: 6
Performance Analyzer selected
STUDENT STUDY AND TASK MANAGER SYSTEM
1. Task Manager
2. Performance Analyzer
3. Search and Sort
4. Exit
enter your choice:|

```

Result - Search & Sort Module

Sample run searching for the task "calc" (found), sorting the task list alphabetically (calc, notes, project), and sorting the marks list in ascending order (80.0, 80.0, 90.0), followed by a clean exit.

```
===== SEARCH & SORT =====
1. Search Task
2. Sort Tasks
3. Sort Marks
4. Back to Main Menu
Enter your choice: 1
Enter task to search: calc
Task found: calc

===== SEARCH & SORT =====
1. Search Task
2. Sort Tasks
3. Sort Marks
4. Back to Main Menu
Enter your choice: 2

===== SORTED TASKS =====
calc
notes
project

===== SEARCH & SORT =====
1. Search Task
2. Sort Tasks
3. Sort Marks
4. Back to Main Menu
Enter your choice: 3

===== SORTED MARKS =====
80.0
80.0
90.0

===== SEARCH & SORT =====
1. Search Task
2. Sort Tasks
3. Sort Marks
4. Back to Main Menu
Enter your choice: 4
Search and Sort selected
STUDENT STUDY AND TASK MANAGER SYSTEM
1. Task Manager
2. Performance Analyzer
3. Search and Sort
4. Exit
enter your choice:4
Thank you for using the system!
```

10. Testing Approach

Testing was carried out manually, module by module, as summarised below.

Task Manager Testing

- Add a task and verify it appears correctly in the task list.
- Call View Tasks when no tasks are present and confirm the "No tasks available." message.
- Delete a valid task by number and confirm it is removed from the list.
- Enter an invalid task number when deleting and confirm the "Invalid task number." message.

Performance Analyzer Testing

- Enter marks for multiple subjects and confirm they are stored in order.
- Check that the calculated average matches a manually computed value.
- Verify the highest and lowest reported marks match the entered data.
- Verify the performance category returned matches the defined threshold ranges.

Search & Sort Testing

- Search for a task that exists (case-insensitive) and confirm a match is reported.
- Search for a task that does not exist and confirm "Task not found." is shown.
- Sort a populated task list and confirm alphabetical ordering.

- Sort a populated marks list and confirm ascending numeric ordering.

Note: testing was performed manually via the console rather than with an automated framework such as unittest/pytest. Adding automated unit tests is listed as a future enhancement (Section 14).

11. Challenges Faced

- **Sharing state across modules:** deciding how `task_manager.py`, `performance.py` and `search_sort.py` should all read and update the same tasks/marks lists without a database required careful thought about function parameters and shared list objects.
 - **Menu navigation logic:** structuring nested while-loops so that each sub-menu correctly returns control to the main menu (rather than exiting the whole program) took several iterations to get right.
 - **Keeping modules independent:** ensuring `performance.py` and `search_sort.py` could be understood and modified without needing to read `task_manager.py` end-to-end, in line with the maintainability goal.
 - **Input validation gaps:** realising, on review, that numeric inputs (marks, task numbers, menu choices) are not currently guarded against non-numeric entries, which is the main outstanding risk identified for this project (see Sections 4 and 14).
-

12. Learnings & Key Takeaways

- Gained practical experience breaking a real-world style problem into independent functional modules.
 - Reinforced how Python lists, loops and conditionals combine to build an interactive, stateful application.
 - Learned to use built-in functions (`sorted()`, `max()`, `min()`) confidently instead of re-implementing standard algorithms, while still understanding what they do internally.
 - Understood, through this project, why input validation and error handling are treated as first-class non-functional requirements rather than an afterthought _a console app with no guards is only one bad input away from crashing.
 - Practised using Git/GitHub to track incremental progress across the project's development.
-

13. Future Enhancements

- **Input validation:** wrap all `int()/float()` conversions in try/except blocks to handle non-numeric input gracefully.
 - **Persistence:** save tasks and marks to a JSON or CSV file on exit and reload them on startup, so data survives between runs.
 - **Task due dates & priority:** extend the task model beyond a plain string to include a due date and priority level, enabling more useful sorting.
 - **Automated unit tests:** add a dedicated test file (e.g. using Python's `unittest`) covering the core functions in each module.
 - **Subject-linked marks:** store marks as `(subject_name, mark)` pairs instead of a flat list of numbers, so performance can be reported per subject rather than only in aggregate.
 - **Logging:** add basic logging of user actions to a file to support the "Logging/Monitoring" non-functional requirement more directly.
-

14. References

- Python Software Foundation, *The Python Standard Library* - docs.python.org/3/library/

- Python Software Foundation, *Built-in Functions (sorted, max, min)* - docs.python.org/3/library/functions.html
- Course material and lecture notes, CSE1021 - Introduction to Problem Solving and Programming
- Project repository: github.com/ashweet26bai10746-coder/Study-manager
- VITyarthi, *Build Your Own Project - General Project Instructions & Submission Guidelines*